Aula 04

Banco de Dados II



Departamento de Computação

ORACLE®

ACADEMY



Consultas Avançadas

Consultas Avançadas

- ***Usando*** os ***operadores*** de ***Conjunto:***
  - Os ***operadores*** de ***conjunto permitem combinar*** as ***linhas retornadas*** por ***duas*** ou ***mais consultas***
  - A ***tabela*** a ***seguir exibe*** os ***quatro operadores*** de ***conjunto***
- ***Restrição:***
  - O ***número*** de ***colunas*** e os ***tipos*** de ***colunas retornados*** pelas ***consultas*** devem ***corresponder***, ***embora*** os ***nomes*** de ***colunas possam*** ser ***diferentes***

Consultas Avançadas

- ***Usando*** os ***operadores*** de ***Conjunto:***

Operator	Description
`UNION ALL`	Returns all the rows retrieved by the queries, including duplicate rows.
`UNION`	Returns all non-duplicate rows retrieved by the queries.
`INTERSECT`	Returns rows that are retrieved by both queries.
`MINUS`	Returns the remaining rows when the rows retrieved by the second query are subtracted from the rows retrieved by the first query.

Consultas Avançadas

- ***Tabela*** de ***Exemplo:***
  - A ***tabela*** **tb_mais_produtos** é ***apresentada abaixo:***

```
CREATE TABLE tb_mais_produtos (
 id_produto              INTEGER,
 id_tipo_produto         INTEGER,
 nm_produto              VARCHAR2(30) NOT NULL,
 disponivel              CHAR(1),
 PRIMARY KEY (id_produto),
 FOREIGN KEY (id_tipo_produto)
        REFERENCES tb_tipos_produtos(id_tipo_produto)
);
```

Consultas Avançadas

- ***Registros*** da ***Tabela tb_mais_produtos:***

```
INSERT INTO tb_mais_produtos (id_produto, id_tipo_produto,
nm_produto, disponivel)
VALUES (1, 1, 'Modern Science', 'Y');

INSERT INTO tb_mais_produtos (id_produto, id_tipo_produto,
nm_produto, disponivel)
VALUES (2, 1, 'Chemistry', 'Y');

INSERT INTO tb_mais_produtos (id_produto, id_tipo_produto,
nm_produto, disponivel)
VALUES (3, NULL, 'Supernova', 'N');
```

Consultas Avançadas

- ***Registros*** da ***Tabela tb_mais_produtos:***

```
INSERT INTO tb_mais_produtos (id_produto, id_tipo_produto,
nm_produto, disponivel)
VALUES (4, 2, 'Lunar Landing', 'N');

INSERT INTO tb_mais_produtos (id_produto, id_tipo_produto,
nm_produto, disponivel)
VALUES (5, 2, 'Submarine', 'Y');

-- commit the transaction
COMMIT;
```

Consultas Avançadas

- ***Registros*** da ***Tabela tb_mais_produtos:***

ID_PRODUTO	ID_TIPO_PRODUTO	NM_PRODUTO	DISPONIVEL
1	1	Modern Science	Y
2	1	Chemistry	Y
3	(null)	Supernova	N
4	2	Lunar Landing	N
5	2	Submarine	Y

Operadores de Conjunto

- ***Usando*** o ***operador*** **UNION ALL**
  - ***Retorna todas*** as ***linhas recuperadas*** pelas ***consultas***, ***incluindo*** as ***linhas duplicadas***
- ***Exemplo:***
  - ***Uso*** do **UNION ALL** ***recuperando todas*** as ***linhas*** de **TB_PRODUTOS** e **TB_MAIS_PRODUTOS**, ***incluindo*** as ***duplicadas***

```
SELECT id_produto, id_tipo_produto, nm_produto
FROM tb_produtos
UNION ALL
SELECT id_produto, id_tipo_produto, nm_produto
FROM tb_mais_produtos;
```

Operadores de Conjunto

- ***Usando*** o ***operador*** **UNION**
  - ***Retorna somente*** as ***linhas*** **NÃO** ***duplicadas recuperadas*** pelas ***consultas***
- ***Exemplo:***
  - ***Uso*** do **UNION**, ***observe*** que as ***linhas duplicadas*** *"Modern Science"* e *"Chemistry"* ***não*** são ***recuperadas***

```
SELECT id_produto, id_tipo_produto, nm_produto
FROM tb_produtos
UNION
SELECT id_produto, id_tipo_produto, nm_produto
FROM tb_mais_produtos;
```

Operadores de Conjunto

- ***Usando*** o ***operador*** **INTERSECT**
  - ***Retorna somente*** as ***linhas recuperadas*** pelas ***duas consultas***
- ***Exemplo:***
  - ***Uso*** do **INTERSECT**, ***observe*** que são ***retornadas*** as ***linhas*** *"Modern Science"* e *"Chemistry"*

```
SELECT id_produto, id_tipo_produto, nm_produto
FROM tb_produtos
INTERSECT
SELECT id_produto, id_tipo_produto, nm_produto
FROM tb_mais_produtos;
```

Operadores de Conjunto

- ***Usando*** o ***operador*** **MINUS**
  - **Retorna** as ***linhas restantes***, quando as ***linhas recuperadas*** pela **SEGUNDA** ***consulta*** são **SUBTRAÍDAS** das ***linhas recuperadas*** pela **PRIMEIRA**
- ***Exemplo:***
  - ***Uso*** do **MINUS**, ***observe*** que as ***linhas*** da **TB_MAIS_PRODUTOS** são ***subtraídas*** da **TB_PRODUTOS** e as ***linhas restantes*** são ***retornadas***

```
SELECT id_produto, id_tipo_produto, nm_produto
FROM tb_produtos
MINUS
SELECT id_produto, id_tipo_produto, nm_produto
FROM tb_mais_produtos;
```

Operadores de Conjunto

- ***Combinando operadores*** de ***conjunto***
  - ***Permite combinar mais*** de ***duas consultas*** com ***vários operadores*** de ***conjunto***, com os ***resultados retornados*** de um ***operador alimentando*** o ***operador seguinte***
  - Por ***padrão***, os ***operadores*** de ***conjunto*** são ***avaliados*** de ***cima*** para ***baixo***, ***todavia***, você ***poderá utilizar parênteses*** para ***alterar*** a ***precedência***

Operadores de Conjunto

- ***Combinando operadores*** de ***conjunto***
  - A ***estrutura*** da ***tabela*** **TB_ALTERACOES_PRODUTOS** é ***apresentada abaixo:***

```
CREATE TABLE tb_alteracoes_produtos (
 id_produto                    INTEGER,
 id_tipo_produto               INTEGER,
 nm_produto                    VARCHAR2(30) NOT NULL,
 descricao                     VARCHAR2(50),
 preco                         NUMBER(5, 2),
 PRIMARY KEY(id_produto),
 FOREIGN KEY(id_tipo_produto)
        REFERENCES tb_tipos_produtos(id_tipo_produto)
);
```

Operadores de Conjunto

- ***Combinando operadores*** de ***conjunto***
  - ***Registros*** da ***tabela*** **TB_ALTERACOES_PRODUTOS:**

```
INSERT INTO tb_alteracoes_produtos (id_produto, id_tipo_produto,
nm_produto, descricao, preco)
VALUES (1, 1, 'Modern Science', 'A description of modern science', 40.00);

INSERT INTO tb_alteracoes_produtos (id_produto, id_tipo_produto,
nm_produto, descricao, preco)
VALUES (2, 1, 'New Chemistry', 'Introduction to Chemistry', 35.00);

INSERT INTO tb_alteracoes_produtos (id_produto, id_tipo_produto,
nm_produto, descricao, preco)
VALUES (3, 1, 'Supernova', 'A star explodes', 25.99);
```

Operadores de Conjunto

- ***Combinando operadores*** de ***conjunto***
  - ***Registros*** da ***tabela*** **TB_ALTERACOES_PRODUTOS:**

```
INSERT INTO tb_alteracoes_produtos (id_produto, id_tipo_produto,
nm_produto, descricao, preco)
VALUES (13, 2, 'Lunar Landing', 'Documentary', 15.99);

INSERT INTO tb_alteracoes_produtos (id_produto, id_tipo_produto,
nm_produto, descricao, preco)
VALUES (14, 2, 'Submarine', 'Documentary', 15.99);

INSERT INTO tb_alteracoes_produtos (id_produto, id_tipo_produto,
nm_produto, descricao, preco)
VALUES (15, 2, 'Airplane', 'Documentary', 15.99);

COMMIT;
```

Operadores de Conjunto

- ***Combinando operadores*** de ***conjunto***
  - ***Registros*** da ***tabela*** **TB_ALTERACOES_PRODUTOS:**

ID_PRODUTO	ID_TIPO_PRODUTO	NM_PRODUTO	DESCRICAO	PRECO
1	1	Modern Science	A description of modern science	40
2	1	New Chemistry	Introduction to Chemistry	35
3	1	Supernova	A star explodes	25.99
13	2	Lunar Landing	Documentary	15.99
14	2	Submarine	Documentary	15.99
15	2	Airplane	Documentary	15.99

Operadores de Conjunto

- ***Combinando operadores*** de ***conjunto***
- ***Exemplo (01):***
  - ***Usa*** o ***operador*** **UNION** para ***combinar*** os ***resultados*** das ***tabelas*** **TB_PRODUTOS** e **TB_MAIS_PRODUTOS**
  - ***Usa*** o ***operador*** **INTERSECT** para ***combinar*** os ***resultados*** do ***operador*** **UNION** ***anterior*** com os ***resultados*** da ***tabela*** **TB_ALTERACOES_PRODUTOS**
  - ***Usa parênteses*** para ***indicar*** a ***ordem*** de ***avaliação***, que é:
    1. **UNION** ***entre*** as ***tabelas*** **TB_PRODUTOS** e **TB_MAIS_PRODUTOS**
    2. A ***operação*** de **INTERSECT**

Operadores de Conjunto

- ***Combinando operadores*** de ***conjunto***
- ***Exemplo (01)*** - *continuação:*

```
(SELECT id_produto, id_tipo_produto, nm_produto
 FROM tb_produtos
 UNION
 SELECT id_produto, id_tipo_produto, nm_produto
 FROM tb_mais_produtos)
INTERSECT
SELECT id_produto, id_tipo_produto, nm_produto
FROM tb_alteracoes_produtos;
```

ID_PRODUTO	ID_TIPO_PRODUTO	NM_PRODUTO
1	1	Modern Science

Operadores de Conjunto

- ***Combinando operadores*** de ***conjunto***
- ***Exemplo (02):***
  - Os ***parênteses*** são ***modificados*** para que a ***operação*** **INTERSECT** seja ***executada primeiro;***
  - ***Observe*** o ***resultado diferente retornado*** pela ***consulta***, em ***comparação*** ao ***exemplo anterior***

Operadores de Conjunto

- ***Combinando operadores*** de ***conjunto***
- ***Exemplo (02):***

```
SELECT id_produto, id_tipo_produto, nm_produto
FROM tb_produtos
UNION
(SELECT id_produto, id_tipo_produto, nm_produto
FROM tb_mais_produtos
INTERSECT
SELECT id_produto, id_tipo_produto, nm_produto
FROM tb_alteracoes_produtos);
```

Operadores de Conjunto

- ***Combinando operadores*** de ***conjunto***
- ***Exemplo (02):***

ID_PRODUTO	ID_TIPO_PRODUTO	NM_PRODUTO
1	1	Modern Science
2	1	Chemistry
3	2	Supernova
4	2	Tank War
6	2	Z Files
7	2	2412: The Return
8	3	Space Force 9
9	3	From Another Planet
10	4	Classical Music
11	4	Pop 3
12	4	Creative Yell
13	(null)	My Front Line

Consultas Avançadas

- **TRANSLATE(x, da_string, para_string)**
  - ***Converte*** as ***ocorrências*** dos ***caracteres*** em "**da_string**" ***encontrados*** em "**x**" nos ***caracteres correspondentes*** em "**para_string**"
- ***Exemplo (01):***
  - Usa **TRANSLATE** para ***deslocar quatro casas*** para a ***direita*** cada ***caractere*** no ***string*** "**MENSAGEM SECRETA**". "**A**" ***torna-se*** "**E**", "**B**" ***torna-se*** "**F**" e ***assim*** por ***diante***

```
SELECT TRANSLATE ('MENSAGEM SECRETA',
                  'ABCDEFGHIJKLMNOPQRSTUVWXYZ',
                  'EFGHIJKLMNOPQRSTUVWXYZABCD')
FROM dual;
```

TRANSLATE('MENSAGEMSECRETA','ABCDEFGHIJKLMNOPQRSTUVWXYZ','EFGHIJKLMNOPQRSTUVWXYZABCD')
QIRWEKIQ WIGVIXE

Consultas Avançadas

- ***Permite passar valores*** de ***coluna*** para **TRANSLATE**
- ***Exemplo (02):***
  - ***Passando*** a ***coluna*** **NM_PRODUTO** para **TRANSLATE**, a qual ***desloca*** as ***letras*** da ***coluna*** **NM_PRODUTO** ***4 casas*** para a ***direita***

```
SELECT id_produto,
        TRANSLATE (nm_produto,
'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz',
'EFGHIJKLMNOPQRSTUVWXYZABCDefghijklmnopqrstuvwxyzabcd')
FROM tb_produtos;
```

Consultas Avançadas

- ***Usa* TRANSLATE** para ***converter números***
- ***Exemplo (03):***
  - ***Pega*** os ***números*** **12345** e ***converte*** **5** em **6**, **4** em **7**, **3** em **8**, **2** em **9** e **1** em **0**

```
SELECT TRANSLATE(12345,
                 54321,
                 67890)
FROM dual;
```

TRANSLATE(12345,54321,67890)
09876

Consultas Avançadas

- **DECODE(*valor*, *valor_pesquisa*, *resultado*, *valor_padrão*)**
  - ***Compara*** "***valor***" com "***valor_pesquisa***". Se os ***valores forem*** **IGUAIS**, **DECODE** ***retorna*** "***resultado***"; caso ***contrário***, "***valor_padrão***" é ***retornado***
  - **DECODE** ***permite executar lógica*** **IF-THEN-ELSE** em **SQL**, ***sem*** a ***necessidade*** de ***usar*** **PL/SQL**
  - ***Cada*** um dos ***parâmetros*** de **DECODE** pode ser ***uma coluna***, ***um valor literal***, uma ***função*** ou uma ***subconsulta***

Consultas Avançadas

- ***Exemplo (01):***
  - ***Uso*** de **DECODE** com ***valores literais***; **DECODE** ***retorna*** 2 (1 é ***comparado*** com 1 e, ***como eles*** são **IGUAIS**, 2 é ***retornado***):

```
SELECT DECODE(1, 1, 2, 3)
FROM dual;
```

DECODE(1,1,2,3)
2

Consultas Avançadas

- ***Exemplo (02):***
  - ***Uso*** de **DECODE** para ***comparar*** 1 com 2 e, ***como eles*** **NÃO** são **IGUAIS**, 3 é ***retornado***

```
SELECT DECODE(1, 2, 1, 3)
FROM dual;
```

DECODE(1,2,1,3)
3

Consultas Avançadas

- ***Exemplo (03):***
  - ***Compara*** a ***coluna*** **DISPONIVEL**, da ***tabela*** **TB_MAIS_PRODUTOS**; se **DISPONIVEL** for ***igual*** a "Y", o ***string*** "Produto está disponível" é ***retornado***; ***caso contrário***, "Produto não está disponível" é ***retornado***

```
SELECT id_produto, disponivel,
                    DECODE(disponivel, 'Y', 'Produto está disponível',
                                            'Produto não está disponível')
FROM tb_mais_produtos;
```

ID_PRODUTO	DISPONIVEL	DECODE(DISPONIVEL,'Y','PRODUTOESTÁDISPONÍVEL','PRODUTONÃOESTÁDISPONÍVEL')
1	Y	Produto está disponível
2	Y	Produto está disponível
3	N	Produto não está disponível
4	N	Produto não está disponível
5	Y	Produto está disponível

Consultas Avançadas

- ***Exemplo (04):***
  - ***Retorna*** a ***coluna*** **ID_TIPO_PRODUTO** como o ***nome*** do ***tipo*** de ***produto***

```
SELECT id_produto, id_tipo_produto,
        DECODE(id_tipo_produto,
                1, 'Book',
                2, 'Video',
                3, 'DVD',
                4, 'CD',
                   'Magazine')
FROM tb_produtos;
```

ID_PRODUTO	ID_TIPO_PRODUTO	DECODE(ID_TIPO_PRODUTO,1,'BOOK',2,'VIDEO',3,'DVD',4,'CD','MAGAZINE')
6	2	Video
7	2	Video
8	3	DVD
9	3	DVD
10	4	CD
11	4	CD
12	4	CD
13	(null)	Magazine
1	1	Book
2	1	Book
3	2	Video
4	2	Video

Consultas Avançadas

- ***Usando*** a ***expressão*** **CASE**
  - **CASE** ***executa*** a ***lógica*** **IF-THEN-ELSE** em **SQL** (***Oracle 9i*** e ***versões superiores***)
  - ***Semelhante*** à **DECODE**, ***entretanto***, você ***deverá utilizar*** **CASE** ***porque*** ela é ***compatível*** com o ***padrão*** **ANSI** (SQL/92)
  - ***Existem basicamente dois tipos*** de **CASE:**
    - ***Expressões CASE simples***, que ***usam expressões*** para ***determinar*** o ***valor retornado***
    - ***Expressões CASE pesquisadas***, que usam ***condições*** para ***determinar*** o ***valor retornado***

Consultas Avançadas

- ***Usando expressão* CASE *simples***
  - ***Usam expressões incorporadas*** para ***determinar*** o ***valor*** a ser ***retornado***
- ***Sintaxe:***

```
CASE search_expression
    WHEN expression1 THEN result1
    WHEN expression2 THEN result2
    . . .
    WHEN expressionN THEN resultN
    ELSE default_result
END
```

Consultas Avançadas

- ***Usando expressão* CASE *simples***
- ***Exemplo:***
  - ***Apresenta*** uma ***expressão* CASE *simples*** que ***retorna*** os ***tipos*** de ***produto*** com suas ***respectivas descrição***

```
SELECT id_produto, id_tipo_produto,
                        CASE id_tipo_produto
                            WHEN 1 THEN 'Book'
                            WHEN 2 THEN 'Video'
                            WHEN 3 THEN 'DVD'
                            WHEN 4 THEN 'CD'
                            ELSE 'Magazine'
                        END
FROM tb_produtos;
```

Consultas Avançadas

- ***Usando expressão* CASE *pesquisadas***
  - As ***expressões* CASE *pesquisadas*** utilizam ***condições*** para ***determinar*** o ***valor*** de ***retorno***
- ***Sintaxe:***

```
CASE
    WHEN condition1 THEN result1
    WHEN condition2 THEN result2
    . . .
    WHEN conditionN THEN resultN
    ELSE default_result
END
```

Consultas Avançadas

- ***Usando expressão* CASE *pesquisadas***
- ***Exemplo (01):***
  - Usar ***operadores*** em uma ***expressão* CASE *pesquisada***

```
SELECT id_produto, id_tipo_produto,
    CASE
        WHEN id_tipo_produto = 1 THEN 'Book'
        WHEN id_tipo_produto = 2 THEN 'Video'
        WHEN id_tipo_produto = 3 THEN 'DVD'
        WHEN id_tipo_produto = 4 THEN 'CD'
        ELSE 'Magazine'
    END
FROM tb_produtos;
```

Consultas Avançadas

- ***Usando expressão* CASE *pesquisadas***
- ***Exemplo (02):***
  - Usar ***operadores*** em uma ***expressão* CASE *pesquisada***

```
SELECT id_produto, preco,
    CASE
        WHEN preco > 15.00 THEN 'Caro'
        ELSE 'Barato'
    END
FROM tb_produtos;
```

Consultas Hierárquicas

- ***Possibilita organizar dados*** de ***forma hierárquica***
- ***Pessoas*** que ***trabalham*** em uma ***empresa*** (***árvore genealógica*** e as ***peças*** que ***compõem*** um ***motor***)
- ***Exemplo:***
  - ***Hierarquia*** de ***funcionários*** que ***trabalham*** em ***nossa loja***



Consultas Hierárquicas

- ***Tabela*** de ***Exemplo***
  - A ***coluna*** **ID_GERENTE** é uma ***auto-referência*** à ***coluna*** **ID_FUNCIONARIO** da ***tabela*** **TB_MAIS_FUNCIONARIOS**
  - A ***tabela*** **TB_MAIS_FUNCIONARIOS** é ***apresentada abaixo***

```
CREATE TABLE tb_mais_funcionarios (
id_funcionario          INTEGER,
id_gerente              INTEGER,
nome                    VARCHAR2(10) NOT NULL,
sobrenome               VARCHAR2(10) NOT NULL,
cargo                   VARCHAR2(20),
salario                 NUMBER(6, 0),
fg_ativo                INTEGER,
PRIMARY KEY (id_funcionario),
FOREIGN KEY (id_gerente)
        REFERENCES tb_mais_funcionarios(id_funcionario));
```

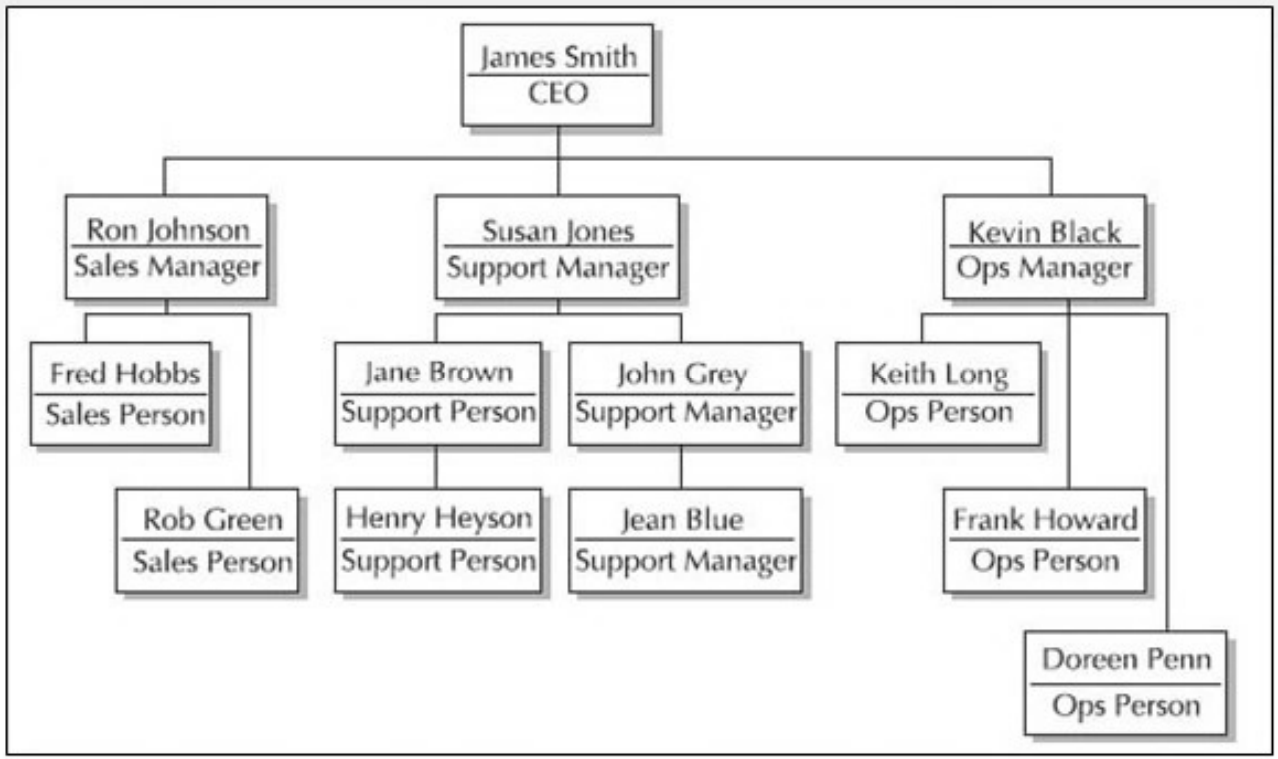
Consultas Hierárquicas

- ***Tabela*** de ***TB_MAIS_FUNCIONARIOS***

ID_FUNCIONARIO	ID_GERENTE	NOME	SOBRENOME	CARGO	SALARIO	FG_ATIVO
1	(null)	James	Smith	CEO	800000	1
2	1	Ron	Johnson	Sales Manager	600000	1
3	2	Fred	Hobbs	Sales Person	200000	1
4	1	Susan	Jones	Support Manager	500000	1
5	2	Rob	Green	Sales Person	40000	1
6	4	Jane	Brown	Support Person	45000	1
7	4	John	Grey	Support Manager	30000	1
8	7	Jean	Blue	Support Person	29000	1
9	6	Henry	Heyson	Support Person	30000	1
10	1	Kevin	Black	Ops Manager	100000	1
11	10	Keith	Long	Ops Person	50000	1
12	10	Frank	Howard	Ops Person	45000	1
13	10	Doreen	Penn	Ops Person	47000	1

Consultas Hierárquicas

- É ***difícil identificar*** os *relacionamentos* dos ***funcionários*** a ***partir*** desses ***dados***
- A ***figura abaixo apresenta*** as ***relações*** de ***forma gráfica***



Consultas Hierárquicas

- ***Usando*** as ***cláusulas*** **CONNECT BY** e **START WITH**
- A ***sintaxe*** das ***cláusulas*** **CONNECT** e **START WITH** de uma ***instrução*** **SELECT** é:

SELECT [**LEVEL**], column, expression, ...
FROM table
[**WHERE** *where_clause*]
[[**START WITH** *start_condition*]
[**CONNECT BY PRIOR** *prior_condition*]];

Consultas Hierárquicas

- ***Usando*** as ***cláusulas*** **CONNECT BY** e **START WITH**
- ***Onde:***
  - **LEVEL:** é uma ***pseudocoluna*** que ***indica*** o ***nível*** que você está ***dentro*** de uma ***árvore***. **LEVEL** ***retorna*** **1** para o ***nó raiz***, **2** para ***um filho*** da ***raiz*** e ***assim por diante***
  - **condição_inicial:** ***especifica*** onde ***iniciar*** a ***consulta hierárquica***. Você deve ***especificar*** uma ***cláusula*** **START WITH** quando ***escrever*** uma ***consulta hierárquica***
  - ***Exemplo:*** ***condição_inicial*** é **id_funcionario** = **1**, que ***especifica*** que a ***consulta inicia-se*** no ***funcionário*** **nº 1**

Consultas Hierárquicas

- ***Usando*** as ***cláusulas*** **CONNECT BY** e **START WITH**
- ***Onde*** (*continuação*)***:***
  - **condição_anterior:** ***especifica*** a ***relação*** entre as ***linhas pai*** e ***filho***
  - Você deve ***especificar*** uma ***cláusula*** **CONNECT BY PRIOR** ao ***escrever*** uma ***consulta hierárquica***
  - ***Exemplo*** de ***condição_anterior*** é ***id_funcionario*** = ***id_gerente***, que ***especifica*** que a ***relação*** é ***entre*** o ***id_funcionario (pai)*** e o ***id_gerente (filho)***

Consultas Hierárquicas

- ***Exemplo:***
  - ***Observe*** que a ***primeira linha contém*** os ***detalhes*** de ***James Smith*** (***funcionário*** nº **1**), a ***segunda linha contém*** os ***detalhes*** de ***Ron Johnson***, cujo ***valor*** de ***id_gerente*** é **1**, e ***assim*** por ***diante***

```
SELECT id_funcionario, id_gerente, nome, sobrenome
FROM tb_mais_funcionarios
START WITH id_funcionario = 1
CONNECT BY PRIOR id_funcionario = id_gerente
ORDER BY 1;
```

Consultas Hierárquicas

- ***Exemplo*** (*continuação*)***:***

ID_FUNCIONARIO	ID_GERENTE	NOME	SOBRENOME
1	(null)	James	Smith
2	1	Ron	Johnson
3	2	Fred	Hobbs
4	1	Susan	Jones
5	2	Rob	Green
6	4	Jane	Brown
7	4	John	Grey
8	7	Jean	Blue
9	6	Henry	Heyson
10	1	Kevin	Black
11	10	Keith	Long
12	10	Frank	Howard
13	10	Doreen	Penn

Consultas Hierárquicas

- ***Usando*** a ***pseudocoluna*** **LEVEL**
- ***Exemplo:***
  - ***Uso*** da ***pseudocoluna*** **LEVEL** para ***exibir*** o ***nível*** na ***árvore***

```
SELECT LEVEL, id_funcionario, id_gerente, nome, sobrenome
FROM tb_mais_funcionarios
START WITH id_funcionario = 1
CONNECT BY PRIOR id_funcionario = id_gerente
ORDER BY LEVEL;
```

Consultas Hierárquicas

- ***Usando*** a ***pseudocoluna*** **LEVEL**
- ***Exemplo*** (*continuação*)**:**
  - ***Uso*** da ***pseudocoluna*** **LEVEL** para ***exibir*** o ***nível*** na ***árvore***

LEVEL	ID_FUNCIONARIO	ID_GERENTE	NOME	SOBRENOME
1	1	(null)	James	Smith
2	10	1	Kevin	Black
2	2	1	Ron	Johnson
2	4	1	Susan	Jones
3	13	10	Doreen	Penn
3	7	4	John	Grey
3	11	10	Keith	Long
3	5	2	Rob	Green
3	3	2	Fred	Hobbs
3	12	10	Frank	Howard
3	6	4	Jane	Brown
4	8	7	Jean	Blue
4	9	6	Henry	Heyson

Consultas Hierárquicas

- ***Formatando*** os ***resultados*** de uma ***consulta hierárquica usando*** **LEVEL** e a ***função*** **LPAD**
- **LPAD *preenche*** os ***valores*** com ***caracteres*** a ***esquerda***
- ***Exemplo:***

```
SELECT LEVEL,
   LPAD(' ', 2 * LEVEL - 1) || nome || ' ' || sobrenome AS funcionario
FROM tb_mais_funcionarios
START WITH id_funcionario = 1
CONNECT BY PRIOR id_funcionario = id_gerente;
```

Consultas Hierárquicas

- ***Exemplo*** (*continuação*)**:**

LEVEL	FUNCIONARIO
1	James Smith
2	Ron Johnson
3	Fred Hobbs
3	Rob Green
2	Susan Jones
3	Jane Brown
4	Henry Heyson
3	John Grey
4	Jean Blue
2	Kevin Black
3	Keith Long
3	Frank Howard
3	Doreen Penn



Consultas Avançadas
Cláusula GROUP BY estendida

Cláusula GROUP BY

- ***Usando cláusula* GROUP BY *estendida***
- **ROLLUP**
  - ***Retorna*** uma ***linha contendo*** um ***subtotal*** para ***cada grupo*** de ***linhas***, ***além*** de uma ***linha contendo*** um ***total geral*** para ***todos*** os ***grupos***
- **CUBE**
  - ***Retorna linhas contendo*** um ***subtotal*** para ***todas*** as ***combinações*** de ***colunas***, além de uma ***linha contendo*** o ***total geral***

Cláusula GROUP BY

- As ***seguintes tabelas abaixo refinam*** a ***representação*** de ***funcionários*** em ***nossa loja virtual***
- **TB_DIVISOES**
  - ***Armazena*** as ***divisões*** (***departamentos***) ***dentro*** da ***empresa***
- **TB_CARGOS**
  - ***Armazena*** os ***cargos dentro*** da ***empresa***
- **TB_FUNCIONARIOS_2**
  - ***Armazena*** os ***funcionários***

Cláusula GROUP BY

- A ***tabela*** **TB_DIVISOES** é ***criada usando*** a ***seguinte instrução:***

```
CREATE TABLE tb_divisoes (
id_divisao        CHAR(3),
nm_divisao        VARCHAR2(15) NOT NULL,
PRIMARY KEY (id_divisao)
);
```

ID_DIVISAO	NM_DIVISAO
SAL	Sales
OPE	Operations
SUP	Support
BUS	Business

Cláusula GROUP BY

- A ***tabela*** **TB_CARGOS** é ***criada usando*** a ***seguinte instrução:***

```
CREATE TABLE tb_cargos (
id_cargo        CHAR(3),
nm_cargo        VARCHAR2(20) NOT NULL,
PRIMARY KEY(id_cargo)
);
```

ID_CARGO	NM_CARGO
WOR	Worker
MGR	Manager
ENG	Engineer
TEC	Technologist
PRE	President

Cláusula GROUP BY

- A ***tabela*** **TB_FUNCIONARIOS_2** é ***criada usando*** a ***seguinte instrução:***

```
CREATE TABLE tb_funcionarios_2 (
id_funcionario          INTEGER,
id_divisao              CHAR(3),
id_cargo                CHAR(3),
nome                    VARCHAR2(10) NOT NULL,
sobrenome               VARCHAR2(10) NOT NULL,
salario                 NUMBER(6,0),
PRIMARY KEY(id_funcionario),
FOREIGN KEY(id_divisao) REFERENCES tb_divisoes(id_divisao),
FOREIGN KEY(id_cargo) REFERENCES tb_cargos(id_cargo)
);
```

Cláusula GROUP BY

- A ***tabela*** **TB_FUNCIONARIOS_2** é ***criada usando*** a ***seguinte instrução:***

ID_FUNCIONARIO	ID_DIVISAO	ID_CARGO	NOME	SOBRENOME	SALARIO
1	BUS	PRE	James	Smith	800000
2	SAL	MGR	Ron	Johnson	350000
3	SAL	WOR	Fred	Hobbs	140000
4	SUP	MGR	Susan	Jones	200000
5	SAL	WOR	Rob	Green	350000
6	SUP	WOR	Jane	Brown	200000
7	SUP	MGR	John	Grey	265000
8	SUP	WOR	Jean	Blue	110000
9	SUP	WOR	Henry	Heyson	125000
10	OPE	MGR	Kevin	Black	225000
11	OPE	MGR	Keith	Long	165000
12	OPE	WOR	Frank	Howard	125000
13	OPE	WOR	Doreen	Penn	145000
14	BUS	MGR	Mark	Smith	155000

Cláusula ROLLUP

- **ROLLUP *estende* GROUP BY** para ***retornar*** uma ***linha contendo*** um ***subtotal*** para ***cada grupo*** de ***linhas***
- ***Retorna também*** uma ***linha contendo*** um ***total*** para ***todos*** os ***grupos***

Cláusula ROLLUP

- ***Exemplo:***
  - ***Utilizando* GROUP BY** para ***agrupar*** as ***linhas*** da ***tabela* TB_FUNCIONARIOS_2** por **ID_DIVISAO**, ***juntamente*** com a ***função* SUM** para ***obter*** a ***soma*** dos ***salários*** para ***cada* ID_DIVISAO**

```
SELECT id_divisao, SUM(salario)
FROM tb_funcionarios_2
GROUP BY id_divisao
ORDER BY id_divisao;
```

ID_DIVISAO	SUM(SALARIO)
BUS	1610000
0PE	1320000
SAL	4936000
SUP	1015000

Cláusula ROLLUP

- ***Passando*** uma ***única coluna*** para **ROLLUP**
- ***Exemplo:***
  - ***Reescreve*** o ***exemplo anterior*** para ***usar*** **ROLLUP**
  - ***Observe*** a ***linha final***, a qual ***contém*** os ***totais*** de ***salários*** de ***todos*** os ***grupos***

```
SELECT id_divisao, SUM(salario)
FROM tb_funcionarios_2
GROUP BY ROLLUP (id_divisao)
ORDER BY id_divisao;
```

ID_DIVISAO	SUM(SALARIO)
BUS	1610000
OPE	1320000
SAL	4936000
SUP	1015000
(null)	8881000

Cláusula ROLLUP

- ***Passando várias colunas*** para **ROLLUP**
- ***Exemplo:***
  - ***Passa*** as ***colunas*** **ID_DIVISAO** e **ID_CARGO** da ***tabela*** **TB_FUNCIONARIOS_2** para **ROLLUP**, o qual ***agrupa*** as ***linhas*** por ***essas colunas***
  - Na ***saída***, ***observe*** que os ***salários*** são ***somados*** por **ID_DIVISAO** e **ID_CARGO**, onde **ROLLUP** ***retorna*** uma ***linha*** com a ***soma*** dos ***salários*** em ***cada*** **ID_DIVISAO**, ***além*** de uma ***linha*** no ***final*** com o ***total*** geral dos ***salários***

Cláusula ROLLUP

- ***Passando várias colunas*** para **ROLLUP**
- ***Exemplo:***

```
SELECT id_divisao, id_cargo, SUM(salario)
FROM tb_funcionarios_2
GROUP BY ROLLUP (id_divisao, id_cargo)
ORDER BY id_divisao, id_cargo;
```

ID_DIVISAO	ID_CARGO	SUM(SALARIO)
BUS	MGR	530000
BUS	PRE	800000
BUS	WOR	280000
BUS	(null)	1610000
OPE	ENG	245000
OPE	MGR	805000
OPE	WOR	270000
OPE	(null)	1320000

Cláusula CUBE

- **CUBE *estende* GROUP BY** para ***retornar*** as ***linhas*** que ***contêm*** um ***subtotal*** para ***todas*** as ***combinações*** de ***colunas***, ***além*** de uma ***linha contendo*** o ***total geral***
- ***Exemplo:***
  - ***Passa* ID_DIVISAO** e **ID_CARGO** para **CUBE**, o qual ***agrupa*** as ***linhas*** por ***essas colunas***

```
SELECT id_divisao, id_cargo, SUM(salario)
FROM tb_funcionarios_2
GROUP BY CUBE(id_divisao, id_cargo)
ORDER BY id_divisao, id_cargo;
```

Cláusula CUBE

- ***Exemplo*** (*continuação*)***:***
  - ***Observe*** que os ***salários*** são ***somados*** por **ID_DIVISAO** e **ID_CARGO**
  - **CUBE *retorna*** uma ***linha*** com a ***soma*** dos ***salários*** para cada **ID_DIVISAO**, junto com a ***soma*** de ***todos*** os ***salários*** para cada **ID_CARGO** ao ***final***, ***juntamente*** com ***uma linha contendo*** o ***total geral*** dos ***salários***

Cláusula CUBE

- ***Exemplo*** (*continuação*)***:***

ID_DIVISAO	ID_CARGO	SUM(SALARIO)
BUS	MGR	530000
BUS	PRE	800000
BUS	WOR	280000
BUS	(null)	1610000
OPE	ENG	245000
OPE	MGR	805000
OPE	WOR	270000
OPE	(null)	1320000
SAL	MGR	4446000
SAL	WOR	490000

ID_DIVISAO	ID_CARGO	SUM(SALARIO)
SAL	(null)	4936000
SUP	MGR	465000
SUP	TEC	115000
SUP	WOR	435000
SUP	(null)	1015000
(null)	ENG	245000
(null)	MGR	6246000
(null)	PRE	800000
(null)	TEC	115000
(null)	WOR	1475000
(null)	(null)	8881000

Função GROUPING

- A ***função*** **GROUPING** ***aceita*** uma ***coluna*** e ***retorna*** **0** (***zero***) ou **1** (***um***)
  - ***Retorna*** **1** ***quando*** o ***valor*** da ***coluna*** é **NULO**
  - ***Retorna*** **0** ***quando*** o ***valor*** da ***coluna*** **NÃO** é **NULO**
  - **GROUPING** ***somente*** é ***utilizada*** em ***consultas*** que ***utilizam*** **ROLLUP**

Função GROUPING

- ***Usando* GROUPING** com ***uma única coluna*** em **ROLLUP**
- ***Exemplo:***
  - ***Observe*** que **GROUPING *retorna* 0 *para*** as ***linhas*** que ***têm valores*** de **ID_DIVISAO *não nulos*** e ***retorna* 1** para a ***última linha***, que ***possui*** um ***valor*** de **ID_DIVISAO *nulo***

```
SELECT GROUPING(id_divisao), id_divisao, SUM(salario)
FROM tb_funcionarios_2
GROUP BY ROLLUP(id_divisao)
ORDER BY id_divisao;
```

GROUPING(ID_DIVISAO)	ID_DIVISAO	SUM(SALARIO)
0	BUS	1610000
0	OPE	1320000
0	SAL	4936000
0	SUP	1015000
1	(null)	8881000

Função GROUPING

- ***Usando* CASE** para ***converter*** o ***valor retornado*** de **GROUPING**
- ***Exemplo:***
  - **CASE *converte* 1** no ***string*** "*Todas as Divisões*"

```
SELECT
   CASE GROUPING(id_divisao)
      WHEN 1 THEN 'Todas as Divisões'
      ELSE id_divisao
   END AS DIV, SUM(salario)
FROM tb_funcionarios_2
GROUP BY ROLLUP(id_divisao)
ORDER BY id_divisao;
```

DIV	SUM(SALARIO)
BUS	1610000
OPE	1320000
SAL	4936000
SUP	1015000
Todas as Divisões	8881000



DML
Alterando o conteúdo das Tabelas

Alterando o conteúdo das Tabelas

- ***Tópicos:***
  - ***Adicionar***, ***modificar*** e ***remover linhas usando*** as ***instruções*** **INSERT**, **UPDATE** e **DELETE**
  - ***Transações constituídas*** por ***várias instruções*** **INSERT**, **UPDATE** e **DELETE**
  - ***Tornando*** os ***resultados*** das ***transações permanentes*** (**COMMIT**) ou ***desfazer*** os ***resultados inteiramente*** (**ROLLBACK**)
  - ***Processando*** "**n**" ***transações simultaneamente*** no ***Oracle***
  - ***Consultas flashback*** (***visualizar registros originalmente***)

Instrução INSERT

- ***Usada*** para ***adicionar linhas*** em ***uma tabela***
- ***Especificação:***
  - A ***tabela*** na qual a ***linha*** será ***inserida***
  - ***Uma lista*** de ***colunas*** para as quais se ***deseja especificar valores***
  - ***Uma lista*** de ***valores*** para ***armazenar*** nas ***colunas especificadas***
- Ao ***adicionar*** uma ***linha***, normalmente ***fornecemos um valor*** para a ***chave primária*** e ***todas*** as ***outras definidas*** como **NOT NULL**
- ***Não*** é ***necessário especificar valores*** para ***colunas*** **NULL**

Instrução INSERT

- O ***Comando*** **DESCRIBE** ***auxilia*** na ***identificação*** de ***colunas definidas*** como **NOT NULL**

```
DESCRIBE tb_clientes
Name           Null     Type
-------------- -------- ------------
ID_CLIENTE     NOT NULL NUMBER(38)
NOME           NOT NULL VARCHAR2(10)
SOBRENOME      NOT NULL VARCHAR2(10)
DT_NASCIMENTO           DATE
TELEFONE                VARCHAR2(12)
FG_ATIVO                NUMBER(38)
```

- As ***colunas*** **ID_CLIENTE, NOME** e **SOBRENOME** são **NOT NULL** (***exige valor***)
- As ***colunas*** **DT_NASCIMENTO, TELEFONE** e **FG_ATIVO** ***não exige valor*** (**NULL**)

Instrução INSERT

- ***Exemplo:***
  - A ***instrução*** **INSERT** ***adiciona uma linha*** na **TB_CLIENTES**
  - ***Observe*** que a ***ordem*** dos ***valores*** na ***cláusula*** **VALUES** ***corresponde*** à ***ordem*** na ***qual*** as ***colunas*** são ***especificadas*** na ***lista*** de ***colunas***

```
INSERT INTO tb_clientes (id_cliente,
                         nome,
                         sobrenome,
                         dt_nascimento,
                         telefone,
                         fg_ativo)
VALUES
(7, 'Joaquim', 'Silva', '26-FEB-1977', '800-666-2522', 1);
```

Instrução INSERT

- ***Observe*** a ***nova linha inserida*** na **TB_CLIENTES**

```
SELECT *
FROM tb_clientes;
```

ID_CLIENTE	NOME	SOBRENOME	DT_NASCIMENTO	TELEFONE	FG_ATIVO
7	Joaquim	Silva	26-FEB-77	800-666-2522	1
1	John	Brown	01-JAN-65	800-555-1211	1
2	Cynthia	Green	05-FEB-68	800-555-1212	1
3	Steve	White	16-MAR-71	800-555-1213	1
4	Gail	Black	(null)	800-555-1214	1
5	Doreen	Blue	20-MAY-70	(null)	1
6	Fred	Brown	01-JAN-70	800-555-1215	1

Instrução INSERT

- ***Omitindo*** a ***lista*** de ***colunas*** ao ***fornecer valores*** para ***cada coluna***

- ***Quando omitimos*** a ***lista*** de ***colunas***, a ***ordem*** dos ***valores fornecidos*** deve ***corresponder*** à ***ordem*** das ***colunas conforme listadas*** na ***saída*** do ***comando*** **DESCRIBE**

Instrução INSERT

- ***Exemplo:***

```
INSERT INTO tb_clientes
VALUES (8, 'Jane', 'Green', '01-JAN-1970', '800-5559999', 1);
```

ID_CLIENTE	NOME	SOBRENOME	DT_NASCIMENTO	TELEFONE	FG_ATIVO
7	Joaquim	Silva	26-FEB-77	800-666-2522	1
8	Jane	Green	01-JAN-70	800-5559999	1
1	John	Brown	01-JAN-65	800-555-1211	1
2	Cynthia	Green	05-FEB-68	800-555-1212	1
3	Steve	White	16-MAR-71	800-555-1213	1
4	Gail	Black	(null)	800-555-1214	1
5	Doreen	Blue	20-MAY-70	(null)	1
6	Fred	Brown	01-JAN-70	800-555-1215	1

Instrução INSERT

- É ***possível especificar*** um ***valor*** **NULO** para uma ***coluna usando*** a ***palavra-chave*** **NULL**
- ***Exemplo:***
  - A ***instrução*** **INSERT** ***especifica*** um ***valor*** **NULO** para as ***colunas*** **DT_NASCIMENTO** e **TELEFONE**

```
INSERT INTO tb_clientes
VALUES (9, 'Sophie', 'White', NULL, NULL, 1);
```

ID_CLIENTE	NOME	SOBRENOME	DT_NASCIMENTO	TELEFONE	FG_ATIVO
7	Joaquim	Silva	26-FEB-77	800-666-2522	1
8	Jane	Green	01-JAN-70	800-5559999	1
9	Sophie	White	(null)	(null)	1
1	John	Brown	01-JAN-65	800-555-1211	1
2	Cynthia	Green	05-FEB-68	800-555-1212	1
3	Steve	White	16-MAR-71	800-555-1213	1
4	Gail	Black	(null)	800-555-1214	1
5	Doreen	Blue	20-MAY-70	(null)	1
6	Fred	Brown	01-JAN-70	800-555-1215	1

Instrução INSERT

- ***Incluindo apóstrofos*** e ***aspas*** em um ***valor*** de ***colunas***
- ***Exemplo:***
  - A ***instrução*** **INSERT** ***especifica*** o **SOBRENOME** O'Malley para o ***novo cliente***

```
INSERT INTO tb_clientes
VALUES (10, 'Kyle', 'O''Malley', NULL, NULL, 1);
```

ID_CLIENTE	NOME	SOBRENOME	DT_NASCIMENTO	TELEFONE	FG_ATIVO
10	Kyle	O'Malley	(null)	(null)	1

Instrução INSERT

- ***Incluindo apóstrofos*** e ***aspas*** em um ***valor*** de ***colunas***
- ***Exemplo:***
  - A ***instrução*** **INSERT** ***especifica*** o **NOME** The “Great” Gatsby para o ***novo produto***

```
INSERT INTO tb_produtos (id_produto,
                         id_tipo_produto,
                         nm_produto,
                         ds_produto,
                         preco,
                         fg_ativo)
VALUES
(14, 1, 'The "Great" Gatsby', NULL, 12.99, 1);
```

ID_PRODUTO	ID_TIPO_PRODUTO	NM_PRODUTO	DS_PRODUTO	PRECO	FG_ATIVO
14	1	The "Great" Gatsby	(null)	12.99	1

Instrução INSERT

- ***Copiando linhas*** de ***uma tabela*** para ***outra usando*** uma ***consulta*** no ***lugar*** dos ***valores***
- O ***número*** de ***colunas*** e os ***tipos*** de ***coluna origem*** e ***destino devem corresponder***

Instrução INSERT

- ***Exemplo:***
  - ***Recuperar*** as ***colunas*** **NOME** e **SOBRENOME** do ***cliente*** nº 1 e ***fornecer*** essas ***colunas*** para a ***instrução*** **INSERT**

```
INSERT INTO tb_clientes (id_cliente,
                         nome,
                         sobrenome)
SELECT 11, nome, sobrenome
FROM tb_clientes
WHERE id_cliente = 1;
```

Instrução UPDATE

- ***Utilizada*** para ***modificar linhas*** em ***uma tabela***
- ***Especificação:***
  - O ***nome*** da ***tabela***
  - Uma ***cláusula*** **WHERE** ***determinando*** as ***linhas*** a ***serem alteradas***
  - ***Uma lista*** de ***nomes*** de ***colunas***, ***juntamente*** com seus ***novos valores***, ***especificados*** na ***cláusula*** **SET**
- ***Podemos alterar uma*** ou ***mais linhas*** por ***meio*** da ***instrução*** **UPDATE**

Instrução UPDATE

- ***Exemplo* (A):**
  - ***Configurar*** a ***coluna*** **SOBRENOME** como "***Orange***" para a ***linha cujo valor*** do **ID_CLIENTE** seja ***equivalente*** ao **nº** 2

```
UPDATE tb_clientes
 SET sobrenome = 'Orange'
WHERE id_cliente = 2;
```

ID_CLIENTE	NOME	SOBRENOME	DT_NASCIMENTO	TELEFONE	FG_ATIVO
2	Cynthia	Orange	05-FEB-68	800-555-1212	1

Instrução UPDATE

- ***Alterando várias linhas*** e ***várias colunas*** na ***mesma instrução***
- ***Exemplo*** (**B**)**:**
  - ***Elevar*** em **20%** o ***preço*** de ***todos*** os ***produtos*** cujo ***preço atual*** seja ***superior*** a **R$ 20,00**
  - ***Alterar*** os ***nomes*** desses ***produtos*** para ***minúsculas***

```
UPDATE tb_produtos
SET
  preco = preco * 1.20,
  nm_produto = LOWER(nm_produto)
WHERE preco > 20.00;
```

Cláusula RETURNING

- ***Oracle 10g*** e ***versões superiores***, a ***cláusula*** **RETURNING** pode ser ***utilizada*** para ***retornar*** o ***valor*** de uma ***função agregada***, como, ***por exemplo***, **AVG**
- ***Exemplo:***
  - ***Declara*** uma ***variável chamada*** **media_preco_produto**
  - ***Decrementa*** o ***valor*** da ***coluna*** **PRECO** das ***linhas*** na **TB_PRODUTOS** e ***armazena*** o ***preço médio*** na ***variável*** **media_preco_produto** ***usando*** a ***cláusula*** **RETURNING**
  - ***Exibe*** o ***valor*** da ***variável*** **media_preco_produto**

Cláusula RETURNING

- ***Exemplo:***

```
DECLARE media_preco_produto NUMBER;

BEGIN
    UPDATE tb_produtos
        SET preco = preco * 0.75
        RETURNING AVG(preco) INTO media_preco_produto;

DBMS_OUTPUT.PUT_LINE('Valor  da variável: ' || media_preco_produto);
END;
```

Instrução DELETE

- ***Usada*** para ***remover linhas*** de ***uma tabela***
  - Deve-se ***especificar*** uma ***cláusula*** **WHERE** que ***limite*** as ***linhas*** que se ***deseja excluir***
  - Se ***não*** for ***feito isso***, ***todas*** as ***linhas serão excluídas***
- ***Exemplo:***
  - ***Remove*** as ***linhas*** da **TB_CLIENTES** cujo ***valor*** da ***coluna*** **ID_CLIENTE** ***corresponda*** ao **nº** 10

```
DELETE
FROM tb_clientes
WHERE id_cliente = 10;

ROLLBACK;
```



Integridade do BD

Integridade do BD

- ***Quando*** se ***executa*** uma ***instrução*** **DML** (**INSERT**, **UPDATE** ou **DELETE**), por exemplo, o **BD *garante*** que as ***linhas*** nas ***tabelas mantenham*** sua ***integridade***

- Isso ***significa*** que as ***alterações feitas*** nas ***linhas não afetam*** a ***chave primária*** e os ***relacionamentos*** de ***chave estrangeira***

Restrições de PK

- ***Exemplo* (A):**
  - A **PK** da **TB_CLIENTES** é a ***coluna* ID_CLIENTE**, o que ***significa*** que ***todo valor armazenado*** nessa ***coluna*** deve ser ***exclusivo***
  - Se ***tentarmos inserir uma linha*** com um ***valor duplicado*** para a **PK**, o **BD *retornará*** um **ERRO ORA-00001**

```
INSERT INTO tb_clientes (id_cliente, nome,
                         sobrenome,
                         dt_nascimento,
                         telefone,
                         fg_ativo)
VALUES
(1, 'Jason', 'Price', '01-JAN-60', '800-555-1211', 1);
```

Restrições de PK

- ***Exemplo*** **(B):**
  - ***Tentar atualizar*** um ***valor*** de **PK** com um ***valor*** já ***existente*** na ***tabela***, o **BD** ***retornará*** o ***mesmo*** **ERRO**

```
UPDATE tb_clientes
   SET id_cliente = 1
WHERE id_cliente = 2;
```

Restrições de FK

- Um ***relacionamento*** de **FK** é ***aquele*** no ***qual uma coluna*** de ***uma tabela*** é ***referenciada*** em ***outra***
- ***Exemplo*** (**A**)**:**
  - A ***coluna*** **ID_TIPO_PRODUTO** da **TB_PRODUTOS** ***referencia*** a ***coluna*** **ID_TIPO_PRODUTO** da **TB_TIPOS_PRODUTOS**
  - A **TB_TIPOS_PRODUTOS** é ***conhecida*** como a ***tabela pai*** e a **TB_PRODUTOS** é ***conhecida*** como ***tabela filha***, ***refletindo*** a ***dependência*** da ***coluna*** **ID_TIPO_PRODUTO** da **TB_PRODUTOS** em ***relação*** à ***coluna*** **ID_TIPO_PRODUTO** da **TB_TIPOS_PRODUTOS**

Restrições de FK

- Um ***relacionamento*** de **FK** é ***aquele*** no ***qual uma coluna*** de ***uma tabela*** é ***referenciada*** em ***outra***
- ***Exemplo*** (**A**) *continuação*:
  - Se ***tentarmos inserir*** uma ***linha*** na **TB_PRODUTOS** com um ***valor*** de **ID_TIPO_PRODUTO** ***inexistente***, o **BD** ***retornará*** o **ERRO ORA-02291**

Restrições de FK

- Um ***relacionamento*** de **FK** é ***aquele*** no ***qual uma coluna*** de ***uma tabela*** é ***referenciada*** em ***outra***
- ***Exemplo*** (**A**) *continuação*:
  - ***Um*** **ERRO** é ***retornado*** porque ***não existe uma linha*** na **TB_TIPOS_PRODUTOS** cujo ***valor*** de **ID_TIPO_PRODUTO** seja ***equivalente*** ao **nº 6**

```
INSERT INTO tb_produtos(id_produto,  id_tipo_produto, nm_produto,
                        ds_produto,
                        preco,
                        fg_ativo)
VALUES
(15, 6, 'Teste', 'Teste', null, 1);
```

Restrições de FK

- Um ***relacionamento*** de **FK** é ***aquele*** no ***qual uma coluna*** de ***uma tabela*** é ***referenciada*** em ***outra***
- ***Exemplo*** (**B**):
  - ***Atualizar*** o ***valor*** de **ID_TIPO_PRODUTO** de ***uma linha*** na **TB_PRODUTOS** com um ***valor*** de ***chave pai inexistente***, o **BD *retornará*** o ***mesmo*** **ERRO**

```
UPDATE tb_produtos
   SET id_tipo_produto = 6
WHERE id_produto = 1;
```

Restrições de FK

- ***Excluir*** uma ***linha*** na ***tabela pai***, que ***tenha linhas filhas dependentes***, o **BD *retornará*** o **ERRO ORA-02292**
- ***Exemplo*** **(C):**
  - ***Excluir*** da **TB_TIPOS_PRODUTOS** a ***linha cujo valor*** de **ID_TIPO_PRODUTO *corresponda*** ao **nº 1**, o **BD *retornará*** um **ERRO**, pois a **TB_PRODUTOS *contém linhas*** cujo ***valor*** de **ID_TIPO_PRODUTO *equivale*** ao **nº 1**

```
DELETE
FROM tb_tipos_produtos
WHERE id_tipo_produto = 1;
```



Valor Padrão

Valor Padrão

- O ***Oracle* 9i *introduziu*** um ***recurso*** que ***permite definir*** um ***valor padrão*** para ***uma coluna***
- ***Exemplo:***
  - ***Criando*** uma ***tabela nomeada*** de **TB_STATUS_ENCOMENDA**
  - A ***coluna* STATUS** tem o ***valor padrão*** "***Encomenda disponibilizada***"
  - A ***coluna* ULTIMA_MODIFICACAO** tem como ***valor padrão*** a ***data/hora retornadas*** por **SYSDATE**

Valor Padrão

- ***Exemplo*** (*continuação*)**:**

```
CREATE TABLE tb_status_encomenda(
id_status_encomenda          INTEGER,
status                       VARCHAR2(40) DEFAULT 'Encomenda
disponibilizada' NOT NULL,
ultima_modificacao           DATE DEFAULT SYSDATE,
PRIMARY KEY (id_status_encomenda)
);
```

Valor Padrão

- ***Exemplo*** (*continuação*)**:**
  - ***Quando adicionamos*** uma ***nova linha*** na **TB_STATUS_ENCOMENDA**, mas ***não especificamos*** os ***valores*** das ***colunas*** **STATUS** e **ULTIMA_MODIFICACAO**, essas ***colunas*** são ***configuradas*** com os ***valores padrões***

```
INSERT INTO tb_status_encomenda(id_status_encomenda)
VALUES (1);

INSERT INTO tb_status_encomenda(id_status_encomenda,
                                status,
                                ultima_modificacao)
VALUES (2, 'Encomenda enviada', '01-MAY-2013');
```

Valor Padrão

- ***Exemplo*:**
  - ***Permite redefinir*** uma ***coluna*** com o ***valor padrão usando*** a ***palavra-chave*** **DEFAULT** em uma ***instrução*** **UPDATE**

```
UPDATE tb_status_encomenda
   SET status = DEFAULT
WHERE id_status_encomenda = 2;
```

ID_STATUS_ENCOMENDA	STATUS	ULTIMA_MODIFICACAO
1	Encomenda disponibilizada	19-JUL-13
2	Encomenda disponibilizada	01-MAY-13



Usando Merge

Merge

- O ***Oracle 9i introduziu*** a ***instrução*** **MERGE**, a qual ***permite mesclar linhas*** de uma ***tabela*** em ***outra***
- ***Exemplo:***
  - ***Possibilidade*** de ***mesclar alterações*** dos ***produtos listados*** em ***uma tabela***, por exemplo, na **TB_PRODUTOS**
  - ***Criando*** a ***tabela*** **TB_PRODUTOS_ALTERADOS**

Merge

- ***Exemplo*** (*continuação*)**:**
  - ***Criando*** a ***tabela*** **TB_PRODUTOS_ALTERADOS**

```
CREATE TABLE tb_produtos_alterados(
id_produto              INTEGER,
id_tipo_produto         INTEGER,
nm_produto              VARCHAR2(30) NOT NULL,
ds_produto              VARCHAR2(50),
preco                   NUMBER(5, 2),
fg_ativo                INTEGER,
PRIMARY KEY (id_produto),
FOREIGN KEY (id_tipo_produto)
      REFERENCES    tb_tipos_produtos(id_tipo_produto)
);
```

Merge

- ***Exemplo*** (*continuação*)**:**
  - ***Inserindo linhas*** na **TB_PRODUTOS_ALTERADOS**

ID_PRODUTO	ID_TIPO_PRODUTO	NM_PRODUTO	DS_PRODUTO	PRECO	FG_ATIVO
1	1	Modern Science	(null)	40	1
2	1	New Chemistry	(null)	35	1
3	1	Super Nova	(null)	25.99	1
13	2	Lunar Landing	(null)	15.99	1
14	2	Submarine	(null)	15.99	1
15	2	Airplane	(null)	15.99	1

Merge

- ***Exemplo:***
  - ***Possibilidade*** de ***mesclar*** as ***linhas*** da **TB_PRODUTOS_ALTERADOS** na **TB_PRODUTOS**
  - ***Para*** as ***linhas*** com ***valores*** de **ID_PRODUTO** ***correspondentes*** nas ***duas tabelas***, ***atualiza*** as ***linhas existentes*** na **TB_PRODUTOS** com os ***valores*** das ***colunas*** da **TB_PRODUTOS_ALTERADOS**
  - O ***produto*** **nº** **1** ***possui*** o ***preço*** em **TB_PRODUTOS_ALTERADOS** ***diferente*** do ***existente*** na **TB_PRODUTOS**

Merge

- ***Exemplo*** (*continuação*)**:**
  - O ***produto*** **nº 2** ***possui*** o ***nome*** e ***preço diferente***, portanto, os ***dois valores precisam*** ser ***atualizados*** na **TB_PRODUTOS**
  - O ***produto*** **nº 3** tem ***um valor*** de **ID_TIPO_PRODUTO** ***diferente*** e, portanto, esse ***valor*** deve ser ***atualizado*** na **TB_PRODUTOS**
  - Para as ***novas linhas*** na **TB_PRODUTOS_ALTERADOS**, ***insere-se*** essas ***novas linhas*** na **TB_PRODUTOS**. Os ***produtos*** de **nº 13**, **14** e **15** são ***novos*** na **TB_PRODUTOS_ALERADOS** e, portanto, ***devem*** ser ***inseridos*** na **TB_PRODUTOS**

Merge

- ***Exemplo*** (*continuação*)**:**
  - ***Realizando*** a ***mesclagem*** conforme ***definido anteriormente***

```
MERGE INTO tb_produtos p
USING tb_produtos_alterados pa ON(p.id_produto = pa.id_produto)
WHEN MATCHED THEN
  UPDATE
  SET
    p.id_tipo_produto = pa.id_tipo_produto,
    p.nm_produto = pa.nm_produto,
    p.ds_produto = pa.ds_produto,
    p.preco = pa.preco,
    p.fg_ativo = pa.fg_ativo      -- continua...
```

Merge

- ***Exemplo*** (*continuação*)**:**
  - ***Realizando*** a ***mesclagem*** conforme ***definido anteriormente***

```
WHEN NOT MATCHED THEN
 INSERT (
  p.id_produto, p.id_tipo_produto, p.nm_produto,
  p.ds_produto, p.preco, p.fg_ativo)
 VALUES (
  pa.id_produto, pa.id_tipo_produto, pa.nm_produto,
  pa.ds_produto, pa.preco, pa.fg_ativo);
```

Merge

- ***Observe*** os ***seguintes aspectos*** sobre a ***instrução*** **MERGE**
  - A ***cláusula*** **MERGE INTO** ***especifica*** o ***nome*** da ***tabela*** a qual as ***linhas*** serão ***mescladas***. No exemplo, essa ***tabela*** é **TB_PRODUTOS**, que ***recebeu*** o ***apelido*** de **p**
  - A ***cláusula*** **USING... ON** ***especifica*** uma ***junção*** das ***tabelas***. No exemplo, a ***junção*** é ***feita através*** das ***colunas*** **ID_PRODUTO** das ***tabelas*** **TB_PRODUTOS** e **TB_PRODUTOS_ALTERADOS**
  - A **TB_PRODUTOS_ALTERADOS** também ***recebeu*** um ***apelido***, **pa**

Merge

- ***Observe*** os ***seguintes aspectos*** sobre a ***instrução*** **MERGE** (*continuação*)
  - A ***cláusula*** **WHEN MATCHED THEN** ***especifica*** a ***ação*** a ser ***executada*** quando a ***cláusula*** **USING... ON** é ***satisfeita*** por ***uma linha***. No ***exemplo***, essa ***ação*** é uma ***instrução*** de **UPDATE** a qual ***configura*** as ***colunas*** **ID_TIPO_PRODUTO**, **NM_PRODUTO**, **DS_PRODUTO**, **PRECO** e **FG_ATIVO** da ***linha existente*** na **TB_PRODUTOS** com os ***valores*** de ***coluna*** da ***linha correspondente*** na **TB_PRODUTOS_ALTERADOS**

Merge

- ***Observe*** os ***seguintes aspectos*** sobre a ***instrução*** **MERGE** (*continuação*)
  - A ***cláusula*** **WHEN NOT MATCHED THEN** ***especifica*** a ***ação*** a ser ***executada*** quando a ***cláusula*** **USING... ON** ***não*** é ***satisfeita*** para ***uma linha***. No ***exemplo***, essa ***ação*** é uma ***instrução*** de **INSERT** a qual ***adiciona*** uma ***linha*** na **TB_PRODUTOS**, pegando os ***valores*** de ***coluna*** da ***linha*** na **TB_PRODUTOS_ALTERADOS**

Merge

- São ***mescladas*** **06** ***linhas***, essas com ***valores*** de **ID_PRODUTO** 1, 2, 3, 13, 14 e 15
- A ***consulta abaixo recupera*** as ***seis linhas mescladas*** da **TB_PRODUTOS**

```
SELECT id_produto, id_tipo_produto, nm_produto, preco
FROM tb_produtos
WHERE id_produto IN (1, 2, 3, 13, 14, 15);
```

Merge

ID_PRODUTO	ID_TIPO_PRODUTO	NM_PRODUTO	PRECO
1	1	Modern Science	40
2	1	New Chemistry	35
3	1	Super Nova	25.99
13	2	Lunar Landing	15.99
14	2	Submarine	15.99
15	2	Airplane	15.99

- O ***produto*** **nº** 1 ***tem*** um ***novo preço***
- O ***produto*** **nº** 2 ***tem*** um ***novo nome*** e ***preço***
- O ***produto*** **nº** 3 ***tem*** uma ***nova identificação*** de ***tipo*** de ***produto***
- Os ***produtos*** de **nº** 13, 14 e 15 ***são novos***



Transações

Transações

- ***Uma transação*** de **BD** é um ***grupo*** de ***instruções*** **SQL** que ***executam*** uma ***unidade lógica*** de ***trabalho***
- ***Podemos considerar*** uma ***transação*** como um ***conjunto inseparável*** de ***instruções*** **SQL** cujos ***resultados devem*** se ***tornar permanentes*** no **BD** como ***um todo*** (ou ***serem desfeitas*** como ***um todo***)

Transações

- ***Exemplo:***
  - ***Uma transferência*** de ***dinheiro*** de uma ***conta bancária*** para ***outra***
  - ***Uma instrução*** de **UPDATE** ***subtrairia*** do ***valor total*** em ***dinheiro*** de ***uma conta*** e ***outra instrução*** **UPDATE** ***adicionaria dinheiro*** na ***outra conta***
  - ***Tanto*** a ***subtração*** como a ***adição devem*** ser ***permanentemente registradas*** no **BD**
  - ***Caso contrário***, o ***dinheiro*** será ***perdido***
  - Se há um ***problema*** na ***transferência*** de ***dinheiro***, então, ***tanto*** a ***subtração*** como a ***adição devem*** ser ***desfeitas***

Transações

- ***Confirmando*** e ***revertendo*** uma ***transação***
- Para ***registrar permanentemente*** os ***resultados obtidos*** pelas ***instruções*** **SQL** em uma ***transação***, ***realize*** um **COMMIT**
- Se for ***necessário desfazer*** os ***resultados***, ***realize*** um **ROLLBACK**, o qual ***permite restaurar todas*** as ***linhas*** como ***eram originalmente***

Transações

- ***Exemplo* (A):**
  - ***Adicionar uma linha*** na **TB_CLIENTES** e ***depois tornar*** a ***alteração permanente executando*** o **COMMIT**

```
INSERT INTO tb_clientes
VALUES
(12, 'Geraldo', 'Henrique', '31-JUL-1977', '800-112233', 1);

COMMIT;
```

Transações

- ***Exemplo* (B):**
  - ***Atualizar*** o ***cliente*** de **nº 1** e na ***sequência desfazer*** a ***alteração executando*** uma ***instrução*** de **ROLLBACK**

```
UPDATE tb_clientes
   SET nome = 'José'
WHERE id_cliente = 1;

SELECT *
FROM tb_clientes
ORDER BY 1;

ROLLBACK;
```

Transações

- ***Iniciando*** e ***terminado*** uma ***transação***
  - ***Uma transação*** é uma ***unidade lógica*** de ***trabalho*** que ***permite segmentar*** suas ***instruções*** **SQL**
  - ***Uma transação*** tem um ***início*** e ***fim***
- A ***transação começa quando*** um dos ***seguintes eventos ocorre:***
  - ***Quando conectamos*** no **BD** e ***executamos*** uma ***instrução*** **DML**
  - ***Uma transação anterior termina*** e ***inserimos outra instrução*** **DML**

Transações

- ***Iniciando*** e ***terminado*** uma ***transação***
- A ***transação começa quando*** um dos ***seguintes eventos ocorre*** (*continuação*)***:***
  - ***Executamos*** uma ***instrução*** de **COMMIT** ou **ROLLBACK**
  - ***Executamos*** uma ***instrução*** **DDL** (**COMMIT** é ***executado automaticamente***)
  - ***Executamos*** uma ***instrução*** **DCL** (**COMMIT** é ***executado automaticamente***)
  - Quando ***executamos*** uma ***instrução*** **DML** que ***falha*** (**ROLLBACK** é ***executado automaticamente***)

Transações

- ***Observações:***
  - ***Não confirmar*** ou ***reverter explicitamente*** suas ***transações*** é uma ***má prática***
  - ***Execute*** uma ***instrução*** de **COMMIT** ou **ROLLBACK** no ***final*** de ***suas transações***

Transações

- ***Savepoints*** (***Pontos*** de ***Salvamento***)
  - ***Permite definir*** um ***savepoint*** em ***qualquer lugar dentro*** de uma ***transação***
  - ***Promove facilidades*** no que diz ***respeito*** a ***reverter alterações*** até esse ***savepoint***
  - ***Savepoints*** são ***úteis*** para ***segmentar transações muito longa***, pois, se ***ocorrer*** um ***erro*** após ter ***definido*** um ***savepoint***, ***não*** é ***necessário reverter*** à ***transação*** até o ***início***
  - Devemos utilizar ***savepoints*** com ***moderação*****:** em vez disso, é ***melhor reestruturar*** sua ***transação*** em ***transações menores***

Transações

- ***Exemplo:***
  - ***Examinar*** o ***preço atual*** dos ***produtos*** de **nº 4** e **nº 6**

```
SELECT id_produto, preco
FROM tb_produtos
WHERE id_produto IN (4, 6);
```

ID_PRODUTO	PRECO
4	10.46
6	44.99

Transações

- ***Exemplo*** (*continuação*)***:***
  - ***Aumentar*** em **20%** o ***preço*** do ***produto*** de **nº 4**

```
UPDATE tb_produtos
   SET preco = preco * 1.20
WHERE id_produto = 4;
COMMIT;
-- A instrução a seguir define um savepoint chamado save1
SAVEPOINT save1;
```

  - ***Todas*** as ***instruções*** **DML** ***executadas posteriormente*** podem ser ***revertidas*** para o ***savepoint*** e a ***alteração feita*** no ***produto*** de **nº** 4 será ***mantida***

Transações

- ***Exemplo*** (*continuação*)***:***
  - ***Aumentar*** em **30%** o ***preço*** do ***produto*** de **nº 6**

```
UPDATE tb_produtos
    SET preco = preco * 1.30
WHERE id_produto = 6;
```

Transações

- ***Exemplo*** (*continuação*)***:***
  - A ***consulta*** a ***seguir obtém*** os ***preços*** dos ***dois produtos***

```
SELECT id_produto, preco
FROM tb_produtos
WHERE id_produto IN (4, 6);
```

ID_PRODUTO	PRECO
4	12.55
6	58.49

Transações

- ***Exemplo*** (*continuação*)***:***
  - A ***instrução*** a ***seguir reverte*** à ***transação*** para o ***savepoint estabelecido anteriormente***

```
ROLLBACK TO SAVEPOINT save1;

SELECT id_produto, preco
FROM tb_produtos
WHERE id_produto IN (4, 6);
```

ID_PRODUTO	PRECO
4	12.55
6	44.99

ACID

- ***Propriedades*** de ***Transação*** (**ACID**)
  - ***Definição*** de uma ***transação*** (***convencional***):
    - ***Unidade lógica*** de ***trabalho***
    - ***Grupo*** de ***instruções*** **SQL** ***relacionadas*** que ***sofrem*** um **COMMIT** ou **ROLLBACK**
- ***Definição adequada***, diz que ***uma transação possui 4 propriedades fundamentais*** (**ACID**)
  - **Atômica:** ***significa*** que as ***instruções*** **SQL** ***contidas*** em uma ***transação constituem*** uma ***única unidade*** de ***trabalho*** (***tudo*** ou ***nada***)

ACID

- ***Definição adequada***, diz que ***uma transação possui 4 propriedades fundamentais*** (**ACID**) - *continuação*
  - **Consistência:** ***significa*** que o **BD** está em um ***estado consistente*** quando uma ***transação inicia***, e, ao seu ***termino***, em ***outro estado consistente***
  - **Isolada:** ***transações separadas não*** devem ***interferir uma*** com a ***outra***
  - **Durável:** uma vez que a ***transação sofreu*** **COMMIT**, as ***alterações feitas*** no **BD** são ***preservadas***, ***mesmo*** que a ***máquina*** em que o **SGBD** ***apresente falha posteriormente***

Transações Concorrentes

- O **SGBD *Oracle*** é ***multiusuário***, ou seja, ***vários usuários*** podem ***interagir*** com o **BD**, e ***cada usuário*** pode ***executar*** suas ***próprias transações simultaneamente*** (***transações concorrentes***)
- Se os ***usuários executarem transações*** que ***afetam*** a ***mesma tabela***, os ***efeitos*** dessas ***transações*** são ***separados*** até que uma ***instrução*** **COMMIT** seja ***executada***

Transações Concorrentes

- A ***sequência*** de ***eventos***, ***baseada*** em ***duas transações nomeadas*** de **T1** e **T2** que ***acessam*** a tabela **CLIENTES**, ilustra a ***separação*** das ***transações*:**

1. **T1** e **T2** ***executam*** uma ***instrução*** de **SELECT** que ***recupera todas*** as ***linhas*** da ***tabela*** **CLIENTES**

2. **T1** ***executa*** uma ***instrução*** **INSERT** para ***adicionar*** uma ***linha*** na ***tabela*** **CLIENTES**, mas ***não executa*** uma ***instrução*** **COMMIT**

Transações Concorrentes

- A ***sequência*** de ***eventos***, ***baseada*** em ***duas transações nomeadas*** de **T1** e **T2** que ***acessam*** a tabela **CLIENTES**, ilustra a ***separação*** das ***transações*** (*continuação*):

3. **T2** ***executa*** outra ***instrução*** **SELECT** e ***recupera*** as ***mesmas linhas*** do ***passo 1***. **T2** ***não*** "***vê***" a ***nova linha adicionada*** por **T1** no ***passo 2***

4. **T1** ***finalmente executa*** uma ***instrução*** **COMMIT** para ***registrar permanentemente*** a ***nova linha adicionada*** no ***passo 2***

Transações Concorrentes

- A ***sequência*** de ***eventos***, ***baseada*** em ***duas transações nomeadas*** de **T1** e **T2** que ***acessam*** a tabela **CLIENTES**, ilustra a ***separação*** das ***transações*** (*continuação*):

5. **T2** ***executa*** outra ***instrução*** **SELECT** e ***finalmente*** "***vê***" a ***nova linha adicionada*** por **T1**

Transações Concorrentes

- ***Podemos simular*** a ***concorrência*** entre ***duas transações*** por ***meio*** de ***duas sessões distintas***

Transação 1 (T1)	Transação 2 (T2)
(1) SELECT * FROM clientes;	**(2)** SELECT * FROM clientes;
(3) INSERT INTO clientes (id_cliente, nm_cliente, sobrenome) VALUES (8, 'Jason', 'Price');	**(6)** SELECT * FROM clientes; O conjunto de resultados retornado não contém a nova linha nem a atualização feita por T1. Em vez disso, o conjunto de resultados contém as linhas originais recuperadas no passo 2
(4) UPDATE clientes SET sobrenome = 'Orange' WHERE id_cliente = 2;	**(8)** SELECT * FROM clientes; O conjunto de resultados retornado contém a nova linha e a atualização feita por T1 nos passos 3 e 4
(5) SELECT * FROM clientes; O conjunto de resultados retornado contém a nova linha e a atualização	
(7) COMMIT; Isso confirma a nova linha e a atualização	

Transações Concorrentes

- ***Observações:***
  - ***Leitores não bloqueiam leitores***
  - ***Gravadores não bloqueiam leitores*** e ***gravadores*** só ***bloqueiam gravadores*** quando eles ***tentam modificar*** a ***mesma linha***

Isolamento de Transação

- ***Nível*** de ***isolamento*** de ***transação:***
  - É o ***grau*** com que as ***alterações feitas*** por ***uma transação*** são ***separadas*** das ***outras transações*** em ***execução concomitante***
  - ***Antes*** de ***evidenciar*** os ***níveis*** de ***isolamento***, é ***necessário entender*** os ***tipos*** de ***problemas*** que ***podem ocorrer*** quando ***transações concorrentes*** tentam ***acessar*** as ***mesmas linhas*** em ***uma tabela***

Isolamento de Transação

- ***Três tipos*** de ***problemas pertinentes*** ao ***processamento*** de ***transação:***
  - **T1** e **T2** (***concorrentes***)
    1. **Leituras Fantasmas**
       - **T1** ***lê*** um ***conjunto*** de ***linhas retornadas*** por ***uma cláusula*** **WHERE** ***especifica***
       - **T2** ***insere*** uma ***nova linha***, a qual ***também satisfaz*** a ***cláusula*** **WHERE** da ***consulta usada anteriormente*** por **T1**
       - **T1** ***lê*** as ***linhas novamente***, ***usando*** a ***mesma consulta***, ***mas agora vê*** a ***linha adicional*** que **T2** ***acabou*** de ***inserir***
       - Essa ***nova linha*** é ***conhecida*** como "***fantasma***" ***porque***, para **T1**, essa ***linha apareceu*** como ***mágica***

Isolamento de Transação

- ***Três tipos*** de ***problemas pertinentes*** ao ***processamento*** de ***transação*** (*continuação*)***:***
  - **T1** e **T2** (***concorrentes***)
    2. **Leituras** que ***não podem*** ser ***repetidas***
       - **T1** ***lê*** uma ***linha*** e **T2** ***atualiza*** a ***mesma linha*** que **T1** ***acabou*** de ***ler***
       - **T1** ***lê novamente*** a ***mesma linha*** e ***descobre*** que a ***linha*** que ***leu anteriormente agora*** está ***diferente***
       - ***Isso*** é ***conhecido*** como ***leitura*** "***que não pode ser repetida***", pois a ***linha lida originalmente*** por **T1** foi ***alterada***

Isolamento de Transação

- ***Três tipos*** de ***problemas pertinentes*** ao ***processamento*** de ***transação*** (*continuação*)***:***
  - **T1** e **T2** (***concorrentes***)
    3. **Leituras *Sujas***
       - **T1** ***atualiza*** uma ***linha***, mas ***não*** ***efetua*** **COMMIT** na ***atualização***
       - **T2** ***lê*** a ***linha atualizada***
       - **T1** ***realiza*** um **ROLLBACK**, ***desfazendo*** a ***atualização anterior***
       - ***Agora*** a ***linha*** que **T2** ***acabou*** de ***ler*** ***não*** é ***mais válida*** (***ela está "suja"***), pois a ***atualização feita*** por **T1** ***não estava confirmada quando*** a ***linha*** foi ***lida*** por **T2**

Isolamento de Transação

- ***Níveis*** de ***Isolamento***
  - ***Utilizadas*** para ***evitar*** que ***transações concorrentes interfiram uma*** com a ***outra***
  - **READ UNCOMMITTED:** ***leituras fantasmas***, ***leituras*** que ***não podem*** ser ***repetidas*** e ***leituras sujas*** são ***permitidas***
  - **READ COMMITTED:** ***leituras fantasmas*** e ***leituras*** que ***não podem*** ser ***repetidas*** são ***permitidas***, ***mas leituras sujas não***

Isolamento de Transação

- ***Níveis*** de ***Isolamento*** (*continuação*)
  - **REPEATABLE READ:** ***leituras fantasmas*** são ***permitidas***, mas ***leituras*** que ***não*** podem ser ***repetidas*** e ***leituras sujas não***
  - **SERIALIZABLE:** ***leituras fantasmas***, ***leituras*** que ***não podem*** ser ***repetidas*** e ***leituras sujas não*** são ***permitidas***

Isolamento de Transação

- ***Oracle suporta apenas*** os ***níveis*** de ***isolamento*** de ***transações*** **READ COMMITTED** e **SERIALIZABLE**
- **SERIALIZABLE**
  - ***Nível*** de ***isolamento*** de ***transação normal definido*** pelo ***padrão*** **SQL** (***padrão usado*** pelo ***Oracle*** – **READ COMMITTED**)

Isolamento de Transação

- **SERIALIZABLE**
  - ***Configurando*** o ***nível isolamento*** de ***transação***
  - ***Exemplo:***

```
SERIALIZABLE SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

Isolamento de Transação

- ***Exemplo*** de ***Transação*** (***isolamento*** **SERIALIZABLE**)
  - **T1** tem o ***nível*** de ***isolamento padrão*** **READ COMMITTED**
  - **T2** tem o ***nível*** de ***isolamento*** **SERIALIZABLE**
  - **T1** e **T2** ***leem*** as ***linhas*** da ***tabela*** **CLIENTES**
  - **T1** ***insere*** uma ***nova linha*** e ***atualiza*** uma ***linha existente*** na ***tabela*** **CLIENTES**
  - **T2** é **SERIALIZABLE**, ***não vê*** a ***linha inserida nem*** a ***atualização feita*** na ***linha existente realizada*** por **T1**, ***mesmo depois*** de **T1** ***efetuar*** o **COMMIT** nas ***alterações***

Isolamento de Transação

- ***Exemplo*** de ***Transação*** (***isolamento*** **SERIALIZABLE**) - *continuação*
  - ***Isso porque ler*** a ***linha inserida seria*** uma ***leitura fantasma*** e ***ler*** a ***atualização seria*** uma ***leitura*** que ***não*** pode ***ser repetida*** (***não permitidas*** pelas ***transações*** **SERIALIZABLE**)



Flashback

Flashback

Você realizou o **COMMIT** em ***algumas alterações***, e na ***verdade***, ***agora***, ***você deseja ver*** as ***linhas*** como ***eram originalmente***, ***o que você pode fazer*** para ***contornar*** esse ***problema***?

Flashback

- ***Possibilidade*** de ***utilizar*** os ***resultados*** da ***consulta flashback permitindo alterar*** as ***linhas manualmente, recuperando*** seus ***valores originais***

- ***Consultas Flashback:***

  - ***Baseadas*** em ***uma data/hora*** ou

  - ***Número*** de ***alteração*** de ***sistema*** (**SCN**)

Flashback

- ***Concedendo*** o ***privilégio*** de ***usar flashbacks:***
  - ***Flashback utiliza*** o ***pacote*** **DBMS_FLASHBACK** (***necessidade*** de ***privilégio*** **EXECUTE**)
  - ***Conecte*** com o ***usuário*** **SYSTEM** e ***conceda*** o ***privilégio*** **EXECUTE** em **DBMS_FLASHBACK** para o ***usuário loja***

```
Enter user-name: system
Enter password:

Connected to:
Oracle Database 11g Enterprise Edition Release 11.2.0.2.0 - Production
With the Partitioning, OLAP, Data Mining and Real Application Testing options

SQL> CONNECT sys/change_on_install AS sysdba
Connected.
SQL> GRANT EXECUTE ON SYS.DBMS_FLASHBACK TO loja;

Grant succeeded.
```

Flashback

- ***Consultas flashback*** de ***tempo***
- ***Exemplo:***
  - ***Recupere*** as ***colunas*** **ID_PRODUTO**, **NM_PRODUTO** e **PRECO** das ***primeiras 5 linhas*** da **TB_PRODUTOS**

```
SELECT id_produto, nm_produto, preco
FROM tb_produtos
WHERE id_produto <= 5;
```

ID_PRODUTO	NM_PRODUTO	PRECO
1	Modern Science	40
2	New Chemistry	35
3	Super Nova	25.99
4	Tank War	12.55

Flashback

- ***Exemplo:***
  - ***Reduzir*** o ***preço*** dessas ***linhas***, ***confirmando*** as ***alterações***

```
UPDATE tb_produtos
    SET preco = preco * 0.75
WHERE id_produto <= 5;

COMMIT;
```

Flashback

- ***Comparando*** os ***resultados***

ID_PRODUTO	NM_PRODUTO	PRECO
1	Modern Science	40
2	New Chemistry	35
3	Super Nova	25.99
4	Tank War	12.55

ID_PRODUTO	NM_PRODUTO	PRECO
1	Modern Science	30
2	New Chemistry	26.25
3	Super Nova	19.49
4	Tank War	9.41

Flashback

- ***Exemplo:***
  - ***Executando*** o ***procedure*** **DBMS_FLASHBACK.ENABLE_AT_TIME**(), que ***permite retornar*** para ***uma data/hora especifica***
  - **24h** x **60 minutos** = **1440 minutos**
  - **SYSDATE – 10/1440** é ***uma data/hora dez minutos*** no ***passado***

```
EXECUTE DBMS_FLASHBACK.ENABLE_AT_TIME(SYSDATE - 10/1440);
```

Flashback

- ***Exemplo:***
  - ***Posteriormente*** a ***execução*** do ***procedure anterior***, ***todas*** as ***consultas*** agora ***exibirão*** as ***linhas*** como ***elas eram*** há ***dez minutos***

```
SELECT id_produto, nm_produto, preco
FROM tb_produtos
WHERE id_produto <= 5;
```

ID_PRODUTO	NM_PRODUTO	PRECO
1	Modern Science	40
2	New Chemistry	35
3	Super Nova	25.99
4	Tank War	12.55

Flashback

- ***Exemplo:***
  - ***Desativando*** o ***flashback***

```
EXECUTE DBMS_FLASHBACK.DISABLE();

SELECT id_produto, nm_produto, preco
FROM tb_produtos
WHERE id_produto <= 5;
```

ID_PRODUTO	NM_PRODUTO	PRECO
1	Modern Science	30
2	New Chemistry	26.25
3	Super Nova	19.49
4	Tank War	9.41

Flashback

- ***Consulta flashback*** com ***número*** de ***alteração*** de ***sistema***
- ***Flashback baseado*** em ***números*** de ***alteração*** de ***sistemas*** (**SCN**s) ***podem*** ser ***mais precisos comparado*** com os ***baseados*** no ***tempo***
- **SCN**s ***são utilizados*** pelo **BD** para ***monitorar*** as ***alterações feitas*** nos ***dados***

Flashback

- ***Exemplo:***
  - ***Usando flashback baseado*** no ***número*** de ***sistema*** (**SCN**)

```
VARIABLE scn_atual NUMBER;

EXECUTE :scn_atual := DBMS_FLASHBACK.GET_SYSTEM_CHANGE_NUMBER();

PRINT scn_atual;
```

```
SCN_ATUAL
---------
13584632
```

Flashback

- ***Exemplo*** (*continuação*)***:***
  - ***Adicionando*** um ***registro*** na **TB_PRODUTOS**, ***efetivando*** a ***inclusão*** e ***recuperando*** o ***novo registro***

```
INSERT INTO tb_produtos(id_produto,
                        id_tipo_produto,
                        nm_produto,
                        ds_produto,
                        preco,
                        fg_ativo)
VALUES
(16, 1, 'Física', 'Livro sobre Física', 39.95, 1);

COMMIT;
```

Flashback

- ***Exemplo*** (*continuação*)***:***
  - ***Executando*** o ***procedure*** o qual ***permite retornar*** a um **SCN** (***variável scn_atual***)
  - As ***consultas agora irão exibir*** as ***linhas*** como ***eram*** no **SCN** ***armazenado*** em ***scn_atual***, ***antes*** da ***execução*** da ***instrução*** **INSERT** (***necessidade*** de ***desativar*** o ***flashback***)

```
EXECUTE DBMS_FLASHBACK.ENABLE_AT_SYSTEM_CHANGE_NUMBER(:scn_atual);

SELECT *
FROM tb_produtos
WHERE id_produto = 16;
```

Flashback

- ***Exemplo (Parte 1):***
  - ***Criando, excluindo*** e ***revertendo*** o ***mesmo objeto através*** do ***uso*** de ***flashback***

```
EXECUTE DBMS_FLASHBACK.DISABLE();

CREATE TABLE tb_teste(
ID              INTEGER,
valor           VARCHAR2(100)
);
```

Flashback

- ***Exemplo (Parte 2):***
  - ***Criando, excluindo*** e ***revertendo*** o ***mesmo objeto através*** do ***uso*** de ***flashback***

```
BEGIN

 FOR v_loop IN 1..100 LOOP
  INSERT INTO tb_teste(id, valor)
  VALUES (v_loop, 'DBA_' || v_loop);
 END LOOP;

END;

SELECT *
FROM tb_teste;
```

Flashback

- ***Exemplo (Parte 3):***
  - ***Criando, excluindo*** e ***revertendo*** o ***mesmo objeto através*** do ***uso*** de ***flashback***

```
DROP TABLE tb_teste; -- equívoco

FLASHBACK TABLE tb_teste TO BEFORE DROP;
```

Exercícios

